TECHNICAL REVIEW DRAFT

Dualscan QR Link SDK Deployment Guide

Mobile deep links for Zelle® QR code URLs

Publisher Dualscan LLC

Prepared for review by Early Warning Services, LLC

13 November 2026 • Draft version 0.1

Dualscan LLC prepared this independent compatibility sample. Zelle® is a trademark of Early Warning Services, LLC. The project has no affiliation with or endorsement from Early Warning Services, LLC or the U.S. Department of the Treasury.

Requested review

Confirm which HTTPS domain and mobile app identities should handle the QR link before a production pilot. For existing codes on enroll.zellepay.com, the domain operator must authorize the intended app through Apple and Android association files. For newly issued pilot codes, a controlled HTTPS host can route the same payload format to the sample apps.

Scope

This manual covers browser QR display, camera and scanner handoff, SDK decoding, domain setup, release checks, and rollback. The SDK exposes the name and recipient token for display. Zelle payment and enrollment remain in the financial institution's supported flow.

Zelle QR code guidance

Guide	Page
Link routes and payload	2
Mobile build and integration	3
HTTPS deployment and verification	4
Device acceptance checks	5
Release operations and rollback	6

02 / OPERATIONS MANUAL

Link routes and payload

The QR symbol contains a complete URL. After recognizing the QR, a camera or scanner offers to open it. The operating system can deliver an eligible HTTPS link directly to an installed app, or the scanner can show it in a browser. The SDK runs only after the app receives the link or the user scans or pastes it inside the app.

Route	Activation	Result
Existing Zelle QR	https://enroll.zellepay.com/qr-codes?data=...	The original QR opens its normal browser destination unless the domain authorizes the receiving app and the device routes the link there. In-app scan or paste can read it directly.
Controlled HTTPS QR	https://qr.example.com/qr-codes?data=...	An installed, verified app can open from a compatible scanner. The hosted page handles browser or uninstalled-app cases. Replace the example host before testing.
Sample custom scheme	dualscan-zelleqr://qr/qr-codes?data=...	Use the browser demo button with the synthetic fixture for local testing. Scanner and browser support varies.

Decoder contract

Each link has one data parameter. Percent-decode its value once, preserving a literal plus sign, then Base64-decode the UTF-8 JSON object. The example uses standard Base64. The decoders also accept Base64URL and optional padding.

```
{"name":"Taylor","token":"12025550123"}
```

Treat both values as strings. A token can contain a recipient identifier, so preserve it without numeric conversion or normalization. The decoders require nonblank fields and enforce the configured host and exact /qr-codes path, with an optional trailing slash.

Malformed input, duplicate data parameters, credentials, explicit ports and URL fragments are rejected. The sample code caps the URL at 8,192 UTF-8 bytes and the decoded query parameter value at 4,096 bytes before Base64 decoding. The payload contains user-supplied data. It provides no proof of enrollment or recipient identity.

Browser QR generation

Encode the same name and token in JSON, Base64-encode the UTF-8 bytes, and percent-encode the result for the query. Use a normal QR symbol with a clear quiet zone for device camera recognition. The browser demo uses synthetic values and generates the QR locally.

03 / OPERATIONS MANUAL

Mobile build and integration

iOS

Open ios/SampleApp/ZelleQRSample.xcodeproj in Xcode and select ZelleQRSample. Choose a development team for device signing. The sample requires iOS 16 or later and uses the local Swift package at ios/ZelleQRLink/. Run swift test from that directory. Use a supported physical device to test camera input.

Add the ZelleQRLink package product to the host app. Pass incoming URLs from SwiftUI onOpenURL to the parser. The sample also accepts a pasted URL and can scan a QR on supported iPhones.

```
let parser = ZelleQRLinkParser(bridgeHost: "qr.example.com",
                               customScheme: "dualscan-zelleqr")
let recipient = try parser.parse(incomingURL)
```

SwiftUI onOpenURL

Android

Open android/ in Android Studio. The sample uses JDK 17 or 21, Android SDK 36 and a minimum device API of 26. Build the sample APK and SDK AAR from the same Gradle project. The delivered APK is debug-signed for evaluation.

```
./gradlew :app:assembleDebug :zelleqrlink:assembleRelease
./gradlew :zelleqrlink:testDebugUnitTest
```

Use the zelleqrlink Gradle module or the AAR with Gson 2.11.0. Forward ACTION_VIEW data from onCreate and onNewIntent to the parser. The sample also supports paste and an in-app scanner on devices with compatible Google Play services.

```
ZelleQrLink parser = new ZelleQrLink("qr.example.com", true);
ZelleQrLink.Recipient recipient = parser.parse(incomingUrl);
```

Local smoke test

Open the supplied browser demo and paste the synthetic canonical link into each app. Confirm Taylor appears, then tap Show token and check for 12025550123. Use the demo custom link to verify delivery on launch and while the app is already running. Manual delivery confirms app handling independently of domain verification.

Apple custom URL schemes

04 / OPERATIONS MANUAL

HTTPS deployment and verification

Configure the receiving apps

Set the same controlled HTTPS host in the iOS sample and Android build. Use the signed identifiers of the apps that should receive links. The checked-in domain files under web/.well-known/ contain placeholders.

Platform	App settings	Server file and values
iOS Universal Links	Set BRIDGE_HOST in SampleSettings.xcconfig and enable CODE_SIGN_ENTITLEMENTS. Use a signing App ID with Associated Domains.	/.well-known/apple-app-site-association. Use the app's application-identifier and scope components to /qr-codes and /qr-codes/.
Android App Links	Set -PqrLinkHost=YOUR_HOST when building. Confirm autoVerify and the two /qr-codes path variants in the merged manifest.	/.well-known/assetlinks.json. Use the installed application ID and SHA-256 signing certificate fingerprint. For Google Play builds, use the Play app signing certificate.

Apple associated domains • Android website association

Publish the web files

Serve both association files with HTTP 200 and application/json over public HTTPS, without redirects. Serve the web fallback for /qr-codes and /qr-codes/ and preserve the data query when routing. Exclude the query from access logs and serve the fallback with a no-referrer policy. Publish a new QR with the controlled HTTPS URL. Existing QR codes retain the URL encoded in their printed or displayed symbol.

Use a regular web fallback when the app is absent or a browser retains navigation. The demo's custom-scheme button is limited to the synthetic fixture. Safari may keep same-domain navigation in the browser. Test links from Camera and a separate source such as Notes.

Apple TN3155 • Apple custom URL schemes

Verify the installed builds

```
curl -i https://YOUR_HOST/.well-known/apple-app-site-association
curl -i https://YOUR_HOST/.well-known/assetlinks.json
```

On iOS, use a freshly installed signed build and confirm a real cross-app tap. On Android 12 or later, reverify the installed app and confirm the intended host is verified. Use an unrestricted VIEW intent when testing default routing.

```
adb shell pm verify-app-links --re-verify com.dualscan.zelleqr.sample
adb shell pm get-app-links com.dualscan.zelleqr.sample
```

Android App Links verification

05 / OPERATIONS MANUAL

Device acceptance checks

Run this matrix on the signed pilot builds and a deployed staging host. Record the OS version, scanner or browser, build identifier, route type, and result. Use the synthetic Taylor fixture for all captured evidence.

Test	Expected behavior
Installed app and verified bridge QR	Tap the recognition result in the default camera or supported scanner. The receiving app shows Taylor. Repeat after terminating the app and while it is already running.
Browser keeps the link or app is absent	The HTTPS route renders the fallback page. Opening the canonical destination remains available.
Original canonical QR	Without an association for the receiving app, the default scanner follows the normal Zelle destination. Pasting or scanning in the sample decodes the same name and token.
Custom scheme on supported clients	Tap the synthetic demo's open button. With the sample installed, the SDK receives the URL and decodes Taylor. Check the browser's behavior without the sample installed.
Negative input	Missing or duplicated data, malformed Base64 or JSON, a wrong host or path, and oversized input produce a clear error. A later valid link decodes successfully.
Scanner unavailable	Paste the recognized URL into the sample. The SDK uses the same decoder.

Diagnose routing failures

Browser opens with app installed. Check scanner behavior, the device's link handling preference, the installed app ID, path, signing identity, and the live association response. On iOS, allow for Apple's association cache. On Android, inspect the per-host verification result.

App opens but the parser rejects the link. Check that QR generation preserves data exactly once through URL encoding. Verify the exact route and confirm the JSON field types using the synthetic fixture. Record an error category in test evidence.

Tap works but a camera scan fails. Test the QR on a second screen or printout. Preserve a clear quiet zone and use an unobstructed symbol. Compare the default camera with the scanner the pilot will support.

06 / OPERATIONS MANUAL

Release operations and rollback

Pilot ownership

Workstream	Requested owner action
Domain operations	Confirm the pilot host and path, publish both association files, maintain HTTPS, and check the fallback after each deploy. If existing Zelle URLs are in scope, confirm who authorizes and publishes the changes.
Mobile release	Record iOS application-identifier, Android application ID and app signing certificate fingerprint. Keep staging associations tied to staging app identities and ship the signed pilot builds after verification.
QA and support	Complete the device matrix before pilot release. Record delivery and parser failures using fixture-based reproduction and escalate routing regressions to domain and mobile owners.

Promote and monitor

Pin the SDK revision and archive the signed build identifiers, checksums and association-file revisions with the release record. Validate public HTTPS responses and the signed builds in staging, then re-run the installed and uninstalled paths after promotion.

Monitor aggregate counts by route, platform, and parser outcome. Omit raw URLs, names, tokens, and the data query from analytics, access logs, referrers, crash reports, and support screenshots. Check this behavior across the CDN and any reverse proxy before pilot traffic.

Restore service

If a release breaks verified routing, pause new QR distribution and restore the last known-good association files and fallback page. Use a staged mobile rollback or route disablement if the app is at fault. Devices can cache associations, so keep a working web path during recovery and retest with fresh installs and already-installed devices. Existing codes retain their original host.

Review record

Record the approved host, app identifiers, signing details, staging evidence, deployment owner, and rollback owner in the release ticket before distributing pilot codes.

Apple TN3155 • Android App Links verification • Zelle QR code guidance